



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Programming

Fachhochschule Oberösterreich

Agenda

-
1. Design Pattern Categories
 2. Selected Design Patterns

Design Patterns

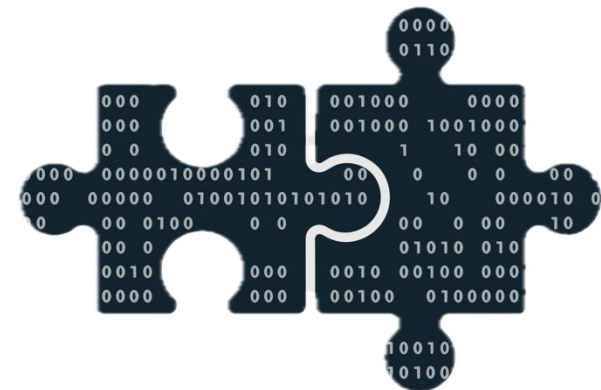
Goals of this Lecture

- Understand design patterns as reusable systems
- Distinguish creational, behavioral, and structural patterns
- Recognize the most useful patterns for game projects
- Apply each pattern to practical game development examples
- Learn when a pattern improves code and when it adds unnecessary complexity

Design Patterns

Design Patterns in Game Programming

- Games are collections of interacting systems
- Recurring problems appear across projects: spawning, state changes, communication, and performance
- Design patterns provide named solutions to these recurring problems
- They are not complete implementations; they are adaptable templates
- The goal is flexible, reusable, and maintainable code



Design Patterns

Software Architecture vs Design Patterns

- **Architecture** concerns the **overall organization** of an application
- **Design patterns** solve **local design problems** inside systems
- Example architecture: client-server multiplayer game
- Example design pattern: Observer for health updates
- Both levels matter in larger game projects

Design Patterns

Pattern Categories

- **Creational** patterns focus on creating objects, initialization, and lifecycle management
 - Examples: Singleton, Factory, Object Pool
- **Behavioral** patterns focus on communication, behavior, and responsibilities
 - Examples: Observer, State, Command
- **Structural** patterns focus on composition, object relationships, and interface compatibility
 - Examples: Adapter, Decorator, Facade

- The pattern *idea* is more important than a textbook-perfect implementation

Design Patterns

Why Use Patterns?

- Games constantly change:
 - Gameplay balancing
 - UI redesign
 - AI updates
 - New features
 - Platform ports
- **Patterns reduce change costs**
- **Reusability**: reuse known solutions
- **Flexibility**: adapt systems to future changes
- **Maintainability**: keep responsibilities clear
- **Communication**: give developers shared vocabulary
- **Scalability**: avoid fragile prototype code in larger projects

Design Patterns

Why Use Patterns?

→ SpaceHuddle: Give some **code reuse** examples in games

Design Patterns

When to Use Patterns

→ Use a pattern when ...

- a concrete design problem exists
- the system is likely to change
- direct references create maintenance costs
- object creation or behavior is becoming complex

→ Avoid patterns when ...

- simple code is clearer

Design Patterns

Design Patterns are Tools

- Not mandatory
- Not magic
- Not always needed
- Solve specific problems
- Should reduce complexity

- **Overengineering:** Too many abstractions can:
 - slow development
 - increase complexity
 - confuse your team members
 - hurt readability

Design Patterns

Common Mistakes

- Using patterns to appear sophisticated
- Adding abstraction before a problem exists
- Mixing too many patterns in one small feature
- Creating patterns that nobody on the team understands
- Ignoring game-engine-specific workflows and Inspector-based composition

Design Patterns

Game Design Patterns

- Singleton
- Factory Method
- Object Pool
- Observer
- Command
- State
- Strategy
- Adapter
- Decorator
- Facade
- Service Locator

Design Patterns

Singleton



→ Purpose:

- Ensure exactly one instance exists
- Provide global access

→ Key idea:

- Static instance
- Global access
- Optional persistence

→ Examples:

- GameManager
- AudioManager
- SaveManager
- SceneLoader
- SettingsManager

Design Patterns

Singleton



→ Basic Singleton

→ Accessing the Singleton

→ Problem:

- This minimal version is not safe against duplicates
- Multiple scenes may create duplicates

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    private void Awake()
    {
        Instance = this;
    }
}
```

```
GameManager.Instance.StartGame();
```

Design Patterns

Singleton



→ Safer Singleton

→ Problem:

- This Singleton is not persisted across Scenes

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
    }
}
```

Design Patterns

Singleton



→ Persistent Singleton

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);
    }
}
```


Design Patterns

Singleton



→ Using the Singleton

```
using UnityEngine;

public class ScorePickup : MonoBehaviour
{
    private void OnTriggerEnter(Collider other)
    {
        if (other.CompareTag("Player"))
        {
            GameManager.Instance.AddScore(100);
            Destroy(gameObject);
        }
    }
}
```

Design Patterns

Singleton



→ Advantages:

- Easy access
- Convenient
- Good for global systems
- Reduces scene references

→ Disadvantages:

- Hidden dependencies
- Harder to test
- Tight coupling
- Often becomes an “EverythingManager”

Design Patterns

Singleton



→ Singleton Overuse

- Singletons are convenient but dangerous when overused
- Create hidden dependencies
- Often used as global dumping grounds
- Can make tests and scene reuse harder
- Not every manager should be globally accessible

- **Instead of one huge *GameManager*, prefer focused managers with clear responsibility:**

- ScoreManager
- LevelManager
- AudioManager
- SaveGameManager
- UIStateController
- GameSessionController

```
using UnityEngine;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    private void Awake()
    {
        Instance = this;
    }
}
```

Design Patterns

Factory



→ Purpose:

- Encapsulate object creation
- Centralize instantiation logic
- Reduce tight coupling between classes
- Allow flexible creation of different object types

→ Key idea:

- The client code does not need to know the exact concrete class
- Object creation becomes reusable and configurable
- Systems become easier to extend

→ Examples:

- Enemy spawning
 - Zombies
 - NPCs
- Weapon creation
 - Swords
 - Guns
 - Loot items
- Wave-based shooters
- RPG enemy generators

Design Patterns

Factory



- The client can depend on an **interface** instead of a concrete enemy class
- This is useful when several enemy **implementations share behavior**

```
public interface IEnemy
{
    void Initialize(int level);
}
```

Design Patterns

Factory



→ Concrete Factory

- The factory owns the prefab reference
- The caller receives an interface reference

```
using UnityEngine;

public class ZombieFactory : MonoBehaviour, IEnemyFactory
{
    [SerializeField]
    private Zombie zombiePrefab;

    public IEnemy CreateEnemy(Vector3 position, int level)
    {
        Zombie enemy = Instantiate(zombiePrefab, position, Quaternion.identity);
        enemy.Initialize(level);
        return enemy;
    }
}
```

Design Patterns

Factory



→ Using the Factory

- The **spawner delegates concrete creation to the factory**
- Changing the factory changes the enemy type without changing the spawner

```
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    [SerializeField]
    private MonoBehaviour factoryBehaviour;
    private IEnemyFactory factory;

    private void Awake()
    {
        factory = (IEnemyFactory)factoryBehaviour;
    }

    public void Spawn(Vector3 position)
    {
        factory.CreateEnemy(position, level: 1);
    }
}
```

Design Patterns

Factory



→ Advantages:

- Centralized creation logic
- Better prefab management
- Cleaner spawning code
- Easier to swap enemy/item families
- Supports testing and balancing workflows

→ Disadvantages:

- Needs serialized prefabs
- More boilerplate code
- Overkill for small projects

Design Patterns

Object Pooling



→ Purpose:

- Reuse objects
- Reduce memory allocations
- Minimize garbage collection
- Improved runtime performance

→ Key idea:

- (Frequently used) Instantiate and Destroy are expensive
- Pooling reuses objects instead of constantly creating them
- Recycling!

→ Examples:

- Bullets
- Particle Effects
- Enemies

Design Patterns

Object Pooling



→ Simple Pool Structure

- Example: Bullets
- The pool pre-instantiates bullets
- Inactive bullets are stored in a queue

```
using System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [SerializeField]
    private GameObject bulletPrefab;

    private Queue<GameObject> bullets = new Queue<GameObject>();

    private void Start()
    {
        for (int i = 0; i < 20; i++)
        {
            GameObject bullet = Instantiate(bulletPrefab);
            bullet.SetActive(false);
            bullets.Enqueue(bullet);
        }
    }

    public GameObject GetBullet()
    {
        GameObject bullet = bullets.Dequeue();
        bullet.SetActive(true);
        return bullet;
    }

    public void ReturnBullet(GameObject bullet)
    {
        bullet.SetActive(false);
        bullets.Enqueue(bullet);
    }
}
```

Design Patterns

Object Pooling



→ Simple Pool Structure

- Example: Bullets
- The pool pre-instantiates bullets
- Inactive bullets are stored in a queue

```
using System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private int initialSize = 30;
    private readonly Queue<GameObject> pool = new();

    private void Awake()
    {
        for (int i = 0; i < initialSize; i++)
            AddBulletToPool();
    }
}
```

Design Patterns

Object Pooling



→ Creating Pooled Objects

- Pooled bullets are **instantiated once**
- They start inactive and are queued for later use

```
using System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private int initialSize = 30;
    private readonly Queue<GameObject> pool = new();

    private void Awake()
    {
        for (int i = 0; i < initialSize; i++)
            AddBulletToPool();
    }

    private GameObject AddBulletToPool()
    {
        GameObject bullet =
            Instantiate(bulletPrefab, transform);
        bullet.SetActive(false);
        pool.Enqueue(bullet);
        return bullet;
    }
}
```

Design Patterns

Object Pooling



→ Retrieving an Object from the Pool

- If the pool is empty, this version expands it
- The returned object is activated for gameplay

```
using System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private int initialSize = 30;
    private readonly Queue<GameObject> pool = new();

    ...

    public GameObject GetBullet()
    {
        if (pool.Count == 0)
        {
            AddBulletToPool();
        }
        GameObject bullet = pool.Dequeue();
        bullet.SetActive(true);
        return bullet;
    }
}
```

Design Patterns

Object Pooling



→ Returning an Object to the Pool

- The object is deactivated and made available again
- A real implementation may also reset velocity, particles, or timers

```
using System.Collections.Generic;
using UnityEngine;

public class BulletPool : MonoBehaviour
{
    [SerializeField] private GameObject bulletPrefab;
    [SerializeField] private int initialSize = 30;
    private readonly Queue<GameObject> pool = new();

    ...

    public void ReturnBullet(GameObject bullet)
    {
        bullet.SetActive(false);
        bullet.transform.SetParent(transform);
        pool.Enqueue(bullet);
    }
}
```

Design Patterns

Object Pooling



→ Advantages:

- Better performance
- Reduced GC spikes
- Stable framerate
- Essential for mobile/VR

→ Disadvantages:

- Pooled objects must be reset correctly
- Pools can hold unused memory
- References may remain in a dirty state
- Pooling everything is unnecessary
- Pool size should be based on expected gameplay demand

Design Patterns

Observer



→ Purpose:

- Decouple communication
- Publish state changes
- Notify listeners automatically

→ Key idea:

- Many systems need to react to a state change
- Instead of directly calling systems ...
 - we **broadcast events**

→ Examples:

- Health UI
- Achievements
- Audio feedback
- Quests
- Score systems

Design Patterns

Observer



→ Components:

- **Subject** (publisher)
 - Owns the state
 - Publishes changes through an event
- **Observer** (listener)
 - UI listens to PlayerHealth
- Event/message

```
using System;
using UnityEngine;

public class PlayerHealth : MonoBehaviour
{
    public event Action<int> OnHealthChanged;

    private int health = 100;

    public void TakeDamage(int damage)
    {
        health -= damage;

        OnHealthChanged?.Invoke(health);
    }
}
```

```
using UnityEngine;
using UnityEngine.UI;

public class HealthUI : MonoBehaviour
{
    [SerializeField]
    private PlayerHealth playerHealth;
    [SerializeField]
    private Text label;

    private void OnEnable()
    {
        playerHealth.OnHealthChanged += UpdateUI;
    }

    private void OnDisable()
    {
        playerHealth.OnHealthChanged -= UpdateUI;
    }

    private void UpdateUI(int health)
    {
        label.text = "Health: " + health;
    }
}
```

- PlayerHealth does not directly know that the UI exists

Design Patterns

Observer



→ Adding another Observer

- Example: DamageAudio
- Another observer can react to the same event
- No change is needed in PlayerHealth

```
using UnityEngine;

public class DamageAudio : MonoBehaviour
{
    [SerializeField]
    private PlayerHealth playerHealth;
    [SerializeField]
    private AudioSource damageSound;

    private void OnEnable()
    {
        playerHealth.OnHealthChanged += PlayDamage;
    }

    private void OnDisable()
    {
        playerHealth.OnHealthChanged -= PlayDamage;
    }

    private void PlayDamage(int value)
    {
        damageSound.Play();
    }
}
```

Design Patterns

Observer



→ Advantages:

- Loose coupling
- Easy extensibility
- Modular systems
- Better reuse

→ Disadvantages:

- Event debugging complexity
- Event leaks
- Forgotten unsubscription
- Too many small events can become event “spaghetti”

Design Patterns

Command



→ Purpose:

- Encapsulate actions
- Separate input from execution

→ Key idea:

- Input should not be hardwired to gameplay behavior
- The same action may be triggered by keyboard, controller, touch, AI, or replay

→ Examples:

- Input remapping
- Replay systems
- AI actions
- Undo systems

Design Patterns

Command



→ Define Interface

```
public interface ICommand
{
    void Execute();
}
```

```
public interface IUndoableCommand : ICommand
{
    void Undo();
}
```

Design Patterns

Command



→ Receiver

- The receiver contains the actual gameplay behavior
- Commands call the receiver instead of implementing all logic themselves

```
using UnityEngine;

public class PlayerMover : MonoBehaviour
{
    public void Move(Vector3 direction)
    {
        transform.position += direction;
    }
}
```

Design Patterns

Command



→ Define Concrete Command

- Example: “Move” Command
- The command packages receiver and movement direction
- It can be reused by player input or AI

```
public interface ICommand
{
    void Execute();
}
```

```
using UnityEngine;

public class MoveCommand : ICommand
{
    private readonly PlayerMover mover;
    private readonly Vector3 direction;

    public MoveCommand(PlayerMover mover, Vector3 direction)
    {
        this.mover = mover;
        this.direction = direction;
    }

    public void Execute()
    {
        mover.Move(direction);
    }
}
```

Design Patterns

Command



→ Input Invoker

- Also called Input Handler
- Input selects which command to execute
- Input logic is separated from movement logic

```
using UnityEngine;

public class PlayerInputInvoker : MonoBehaviour
{
    [SerializeField] private PlayerMover mover;
    private ICommand moveLeft;
    private ICommand moveRight;

    private void Start()
    {
        moveLeft = new MoveCommand(mover, Vector3.left);
        moveRight = new MoveCommand(mover, Vector3.right);
    }

    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.A)) moveLeft.Execute();
        if (Input.GetKeyDown(KeyCode.D)) moveRight.Execute();
    }
}
```


Design Patterns

Command



→ Advantages:

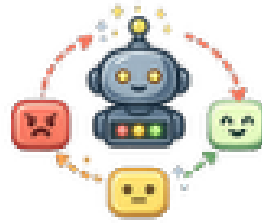
- Input remapping becomes easier
- AI and player can trigger the same actions
- Commands can be logged for replay
- Undo/redo becomes possible

→ Disadvantages:

- Increased complexity:
 - More classes
 - More abstraction
 - More architecture overhead
- Extra indirection:
 - Instead of: *player.FireWeapon()*;
 - You may have: *fireCommand.Execute()*;
which internally calls the weapon system, animation system, sound system etc.

Design Patterns

State



→ Purpose:

- Change behavior dynamically
- Organize gameplay logic

→ Key idea:

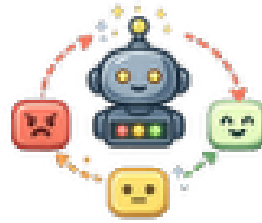
- Object behavior changes over time
- Without states, we would have:
 - giant Update methods
 - many booleans (if/switch statements)
 - tangled gameplay logic

→ Examples:

- Enemy AI
- Menus
- Combat systems
- Animations: Character locomotion

Design Patterns

State



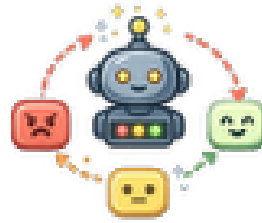
→ State Interface

- Each state supports Enter, Tick, and Exit
- Enter initializes the state
- Tick updates the state
- Exit cleans up before transition

```
public interface IState
{
    void Enter();
    void Tick();
    void Exit();
}
```

Design Patterns

State



→ State Machine

- The state machine owns the current state
- Transitions call Exit and Enter in the correct order

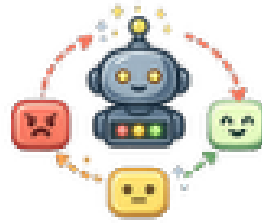
```
public class StateMachine
{
    private IState currentState;

    public void ChangeState(IState newState)
    {
        currentState?.Exit();
        currentState = newState;
        currentState.Enter();
    }

    public void Update()
    {
        currentState?.Tick();
    }
}
```

Design Patterns

State



→ State Implementation

- Example: Enemy Idle State

```
using UnityEngine;

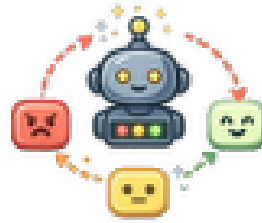
public class EnemyIdleState : IState
{
    private readonly EnemyAI enemy;

    public EnemyIdleState(EnemyAI enemy)
    {
        this.enemy = enemy;
    }

    public void Enter() => Debug.Log("Enemy idle");
    public void Tick() { /* look for player */ }
    public void Exit() { }
}
```

Design Patterns

State



→ State Implementation

- Example: Enemy Attack State
- Attack behavior can be developed independently
- The state machine can switch into this state based on distance or perception

```
using UnityEngine;

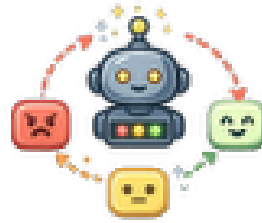
public class EnemyAttackState : IState
{
    private readonly EnemyAI enemy;

    public EnemyAttackState(EnemyAI enemy)
    {
        this.enemy = enemy;
    }

    public void Enter() => Debug.Log("Enemy attacks");
    public void Tick() { /* attack cooldown logic */ }
    public void Exit() { }
}
```

Design Patterns

State



→ State Machine

- Enemy AI Example

```
using UnityEngine;

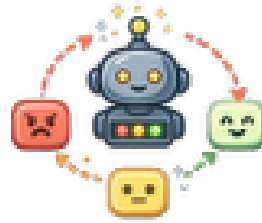
public class EnemyAI : MonoBehaviour
{
    private StateMachine stateMachine;

    private void Start()
    {
        stateMachine = new StateMachine();
        stateMachine.ChangeState(new IdleState());
    }

    private void Update()
    {
        stateMachine.Update();
    }
}
```

Design Patterns

State



→ Advantages:

- Organized logic: behavior is divided into clear classes
- Scalable AI: AI is easier to extend
- Transitions can be tested and logged
- Works well for turn-based systems and menus

→ Disadvantages:

- Too many states
- Transition complexity
- State synchronization issues

Design Patterns

Strategy



→ Purpose:

- An object needs **different** algorithms or behaviors
- Swap algorithms dynamically

→ Key idea:

- Encapsulates interchangeable behavior behind a **common interface**
- Changes behavior without modifying the main object
- Favors composition over inheritance

→ Examples:

- Combat styles
 - MeleeAttack
 - MagicAttack
 - StealthAttack
- AI behaviors
- Movement systems

Design Patterns

Strategy



→ Strategy Interface

- Example: Different Attack Strategies
- The context can call Attack without knowing which algorithm is used.

```
public interface IAttackStrategy
{
    void Attack();
}
```

Design Patterns

Strategy



→ Concrete Strategies

- Melee Strategy
- Ranged Strategy

```
using UnityEngine;

public class MeleeAttack : IAttackStrategy
{
    public void Attack()
    {
        Debug.Log("Melee attack");
    }
}
```

```
using UnityEngine;

public class RangedAttack : IAttackStrategy
{
    public void Attack()
    {
        Debug.Log("Ranged attack");
    }
}
```

Design Patterns

Strategy



→ Enemy Context

```
public class Enemy
{
    private IAttackStrategy strategy;

    public Enemy(IAttackStrategy strategy)
    {
        this.strategy = strategy;
    }

    public void PerformAttack()
    {
        strategy.Attack();
    }
}
```

Design Patterns

Strategy



→ Advantages:

- Avoids large conditional blocks
- Behavior becomes interchangeable
- Good for AI, weapons, movement, and difficulty

→ Disadvantages:

- Many small classes/assets can be hard to organize
- Strategy objects need access to sufficient context
- Not every if-statement deserves a strategy

Design Patterns

Adapter



→ Purpose:

- Converts one interface into another expected interface
- This protects the rest of the game from API changes

→ Key idea:

- Two systems provide similar functionality through **incompatible interfaces**
- Examples: third-party plugins, input APIs, legacy code
- The client should not know every special case

→ Examples:

- Connecting incompatible systems
 - Xbox-specific controls are mapped to a unified game input interface
- Integrating third-party libraries or middleware
 - Audio, networking libs etc.
- Supporting multiple platforms or devices
 - PCs, consoles, Mobile, VR devices

Design Patterns

Adapter



→ Target Interface

- The game code expects this simple interface
- Any input source can be adapted to it

```
using UnityEngine;

public interface IInputSource
{
    Vector2 Move { get; }
    bool JumpPressed { get; }
}
```

Design Patterns

Adapter



→ Keyboard Adapter

- The adapter wraps Unity input calls
- Game code depends on *IInputSource*, not *Input.GetAxisRaw*

```
using UnityEngine;

public class KeyboardInputAdapter : MonoBehaviour, IInputSource
{
    public Vector2 Move => new Vector2(
        Input.GetAxisRaw("Horizontal"),
        Input.GetAxisRaw("Vertical")
    );

    public bool JumpPressed => Input.GetKeyDown(KeyCode.Space);
}
```


Design Patterns

Adapter



→ Using the Adapter

- The movement component does not care where input comes from
- Keyboard, gamepad, AI, or network input can share the same interface

```
using UnityEngine;

public class AdaptedPlayerController : MonoBehaviour
{
    [SerializeField] private MonoBehaviour inputBehaviour;
    private IInputSource input;

    private void Awake() => input = (IInputSource)inputBehaviour;

    private void Update()
    {
        Vector2 move = input.Move;
        if (input.JumpPressed) Debug.Log("Jump");
    }
}
```

Design Patterns

Adapter



→ Advantages:

- Protects game code from external API changes
- Useful for input, analytics, ads, cloud saves, and platform services
- Improves testability through mock adapters
- Makes legacy systems easier to integrate

→ Disadvantages:

- Introduces additional abstraction
- Performance overhead
- Excessive wrapper classes

Design Patterns

Decorator



→ Purpose:

- Wraps an object while preserving a common interface
- Allows behavior to be layered dynamically

→ Key idea:

- Objects sometimes need optional upgrades or modifiers
- Subclassing every combination causes class explosion
 - Examples: weapon buffs, status effects, damage modifiers, and equipment bonuses

→ Examples:

- Weapon dynamically gaining:
 - Fire damage
 - Poison effects
 - Ice effects
 - Critical Hit bonuses
- UI dynamically extending:
 - Glow effects
 - Animations
 - Tooltips

Design Patterns

Decorator



→ Damage Interface

- All damage providers expose the same method
- Decorators can wrap any object implementing this interface

```
public interface IDamageSource
{
    int GetDamage();
}
```

Design Patterns

Decorator



→ Base Weapon

- A concrete component provides base behavior
- Decorators will modify this result

```
public class SwordDamage : IDamageSource
{
    public int GetDamage()
    {
        return 10;
    }
}
```

Design Patterns

Decorator



→ Using Decorators

- The final object still implements IDamageSource
- The client does not know how many decorators were applied

```
using UnityEngine;

public class DecoratorDemo : MonoBehaviour
{
    private void Start()
    {
        IDamageSource sword = new SwordDamage();
        IDamageSource fireSword = new FireDamageDecorator(sword);
        Debug.Log("Damage: " + fireSword.GetDamage());
    }
}
```

Design Patterns

Decorator



→ Advantages:

- Avoids subclass explosion
- Adds optional behavior at runtime
- Good for buffs, modifiers, and RPG systems
- Composable when the interface is stable
- Supports clean feature layering

→ Disadvantages:

- Many small wrapper classes
- Deep decorator chains
- Reduced readability

Design Patterns

Facade



→ Purpose:

- Provides a simple high-level interface
- Hides subsystem coordination from client code

→ Examples:

- Save System
- Menu System

→ Key idea:

- A client often needs to coordinate several subsystems
 - Example: saving may involve inventory, position, progress, and settings
- Calling every subsystem directly spreads complexity

Design Patterns

Facade



→ Concrete Façade

- Example: Save Facade
- The facade exposes one simple method
- Client code does not need to know every subsystem

```
public class SaveGameFacade
{
    private readonly InventorySaveSystem inventory = new();
    private readonly PositionSaveSystem position = new();
    private readonly ProgressSaveSystem progress = new();

    public void SaveGame()
    {
        inventory.SaveInventory();
        position.SavePosition();
        progress.SaveProgress();
    }
}
```

Design Patterns

Facade



→ Advantages:

- Simplifies complex subsystem usage
- Reduces duplicated orchestration logic
- Improves readability for client code
- Can protect client code from subsystem changes
- Useful for save/load, audio setup, networking, and analytics

→ Disadvantages:

- Facade can become a “god” object
- May hide errors from subsystems
- Can introduce an extra layer with little value
- Facade should simplify usage, not own all logic

Design Patterns

Service Locator



→ Purpose:

- Stores services behind interfaces
- Clients request services by **contract**
- Reduces direct dependencies while still offering global access

→ Key idea:

- Some services must be accessible from many places
- Singletons solve access but couple code to concrete classes

→ Examples:

- AudioManager
- Gameplay Service

Design Patterns

Service Locator



→ Service Contract

- Clients depend on an interface
- The concrete service can be swapped

```
public interface IAudioService
{
    void PlaySound(string soundId);
}
```

Design Patterns

Service Locator



→ Concrete Service

- The service implements the contract
- A production version could wrap AudioManager, or FMOD

```
using UnityEngine;

public class UnityAudioService : IAudioService
{
    public void PlaySound(string soundId)
    {
        Debug.Log("Playing sound: " + soundId);
    }
}
```

Design Patterns

Service Locator



→ Basic Service Locator

- Services are stored by type
- Clients retrieve them by interface

```
using System;
using System.Collections.Generic;

public static class ServiceLocator
{
    private static readonly Dictionary<Type, object> services = new();

    public static void Register<T>(T service)
    {
        services[typeof(T)] = service;
    }

    public static T Get<T>()
    {
        return (T)services[typeof(T)];
    }
}
```

Design Patterns

Service Locator



→ Using a Service

- Register services during bootstrap
 - Bootstrap = Initialization of the core systems of the game
- Client code asks for the contract it needs

```
using UnityEngine;

public class GameBootstrap : MonoBehaviour
{
    private void Awake()
    {
        ServiceLocator.Register<IAudioService>(new UnityAudioService());
    }
}
```

```
public class CoinPickup : MonoBehaviour
{
    public void Collect()
    {
        ServiceLocator.Get<IAudioService>().PlaySound("coin");
    }
}
```

Design Patterns

Service Locator



→ Advantages:

- Less concrete coupling than singleton access
- Centralized service registration
- Can support mock services in tests
- Useful for application services such as audio, logging, save, and analytics
- Works well when service scope is well-defined

→ Disadvantages:

- Still creates hidden dependencies
- Can become a global registry for everything
- Order of initialization matters
- Missing services can fail at runtime

Design Patterns

Pattern Selection

- What concrete problem exists?
- Is object creation becoming complicated?
- Is communication too tightly coupled?
- Is behavior changing based on state or configuration?
- Is performance affected by allocation or repeated setup?
- Will this system change during the project?

Design Patterns

Pattern Map

- Creation problem: Factory, Builder, Object Pool
- Communication problem: Observer, Event Channel
- Action binding problem: Command
- Behavior switching problem: State or Strategy
- Interface mismatch: Adapter
- Optional behavior layers: Decorator
- Complex subsystem: Facade
- Global service access: Singleton or Service Locator

Design Patterns

Combining Patterns

- Patterns often work together
- Factory can create pooled objects
- Observer can notify UI about state changes
- Command can trigger state transitions
- Strategy can be selected by a factory
- Facade can coordinate save subsystems

Design Patterns

Example Architecture

- **Factory** creates enemy prefabs
- **State** controls enemy AI mode
- **Strategy** controls attack style
- **Observer** notifies UI and audio about damage
- **Object Pool** reuses projectiles and hit effects
- **Facade** exposes simple combat setup to level scripts

Design Patterns

Summary

- Design patterns are **reusable systems for recurring problems**
- Singleton and Service Locator solve access but must be controlled
- Observer, Command, State, and Strategy are central for gameplay architecture
- Factory and Object Pool are highly practical in Unity and other game engines
- Adapter, Decorator, and Facade improve integration and composition
- Use patterns to **reduce complexity**, not to display complexity

Thanks for your Attention



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at